

System Design

Core Concepts, Common Problems, and Practical Solutions

1. Introduction

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a software system to satisfy specific functional and non-functional requirements. It sits at the intersection of engineering trade-offs: performance versus cost, consistency versus availability, simplicity versus flexibility. A well-designed system is not the one with the most advanced technology, but the one that reliably meets its goals while remaining maintainable and cost-effective as it grows.

This report gives a concise overview of the core building blocks of system design, then walks through the most common problems engineers face when systems scale, along with proven solutions used in real-world architectures.

2. Core Building Blocks

- **Client / Server:** The client sends requests; the server processes them and returns responses. Most web and mobile systems are built on this model.
- **Load Balancer:** Distributes incoming traffic across multiple servers to prevent any single server from becoming a bottleneck.
- **Database:** Stores persistent data. Can be relational (SQL) for structured data and strong consistency, or NoSQL for flexible schemas and horizontal scale.
- **Cache:** A fast, temporary storage layer (e.g., Redis, Memcached) that reduces load on the database and speeds up repeated reads.
- **Message Queue:** Enables asynchronous communication between services (e.g., Kafka, RabbitMQ), decoupling producers from consumers.
- **CDN (Content Delivery Network):** Caches static content closer to users geographically, reducing latency.
- **API Gateway:** A single entry point that routes requests, handles authentication, rate limiting, and logging for backend services.

3. Key Design Principles

Scalability

The ability of a system to handle increasing load by adding resources. Vertical scaling adds power to a single machine; horizontal scaling adds more machines.

Reliability

The system continues to work correctly even when components fail. Achieved through redundancy, replication, and failover mechanisms.

Availability

The percentage of time a system is operational and accessible, often expressed in 'nines' (e.g., 99.99% uptime).

Consistency

All nodes in a distributed system see the same data at the same time. Trade-offs between consistency and availability are formalized in the CAP theorem.

Latency & Throughput

Latency is the time to process a single request; throughput is the number of requests processed per unit of time. Systems are optimized for one or both depending on the use case.

Maintainability

How easily engineers can understand, modify, and extend the system over time without introducing bugs.

4. The CAP Theorem (Summary)

In any distributed data store, it is impossible to simultaneously guarantee Consistency, Availability, and Partition Tolerance. Since network partitions are unavoidable in real systems, architects must choose between prioritizing consistency (CP) or availability (AP) when a partition occurs.

Model	Guarantees	Typical Use Case
CP (Consistency + Partition Tolerance)	Data is always accurate, but some requests may fail during a partition.	Banking transactions, financial records
AP (Availability + Partition Tolerance)	System always responds, but data may be briefly stale.	Social media feeds, shopping carts

5. Common Problems and Their Solutions

Problem: Single Point of Failure (SPOF)

When one component (e.g., a single server or database) fails, the entire system goes down.

Solution: Introduce redundancy — deploy multiple instances behind a load balancer, use database replication (primary-replica setup), and design for automatic failover so no single component can bring down the whole system.

Problem: Database Becomes a Bottleneck

As traffic grows, a single database instance cannot handle the read/write load, causing slow queries and timeouts.

Solution: Use read replicas to offload read traffic, add a caching layer (Redis/Memcached) for frequently accessed data, and apply database sharding to horizontally partition data across multiple database instances.

Problem: High Latency for Global Users

Users far from the server's data center experience slow response times.

Solution: Use a CDN to cache static assets near users, deploy servers in multiple geographic regions, and route users to the nearest region using geo-based DNS or load balancing.

Problem: System Cannot Handle Traffic Spikes

Sudden surges in traffic (e.g., flash sales, viral content) overwhelm fixed server capacity.

Solution: Implement auto-scaling to add/remove servers based on real-time load, use message queues to buffer and smooth bursts of requests, and apply rate limiting to protect backend services.

Problem: Data Inconsistency Across Services

In microservices architectures, each service may have its own database, making it hard to keep data consistent across services.

Solution: Use the Saga pattern for distributed transactions, apply eventual consistency where strict consistency is not required, and use event-driven architecture with a message broker to propagate state changes reliably.

5. Common Problems and Their Solutions (continued)

Problem: Cache Invalidation Issues

Cached data becomes stale, and users see outdated information after an update.

Solution: Use appropriate cache expiration policies (TTL), apply write-through or write-behind caching strategies, and explicitly invalidate or update cache entries when the underlying data changes.

Problem: Difficult to Debug and Monitor Distributed Systems

With many microservices, it becomes hard to trace a request and identify where failures occur.

Solution: Implement centralized logging, distributed tracing (e.g., OpenTelemetry, Jaeger), and real-time monitoring/alerting dashboards to gain visibility across all services.

Problem: Security Vulnerabilities at Scale

More components and public endpoints increase the attack surface of the system.

Solution: Use an API Gateway for centralized authentication and rate limiting, encrypt data in transit and at rest, apply the principle of least privilege, and run regular security audits.

6. Summary Table

Problem	Root Cause	Primary Solution
Single point of failure	No redundancy	Replication + failover
Database bottleneck	One DB handles all load	Caching, replicas, sharding
High latency (global users)	Distance to server	CDN + multi-region deployment
Traffic spikes	Fixed capacity	Auto-scaling + queues
Data inconsistency	Distributed databases	Saga pattern, eventual consistency
Stale cache	No invalidation strategy	TTL + write-through/behind caching
Hard to debug	No visibility	Logging, tracing, monitoring

7. Conclusion

Good system design is fundamentally about managing trade-offs. There is rarely a single 'correct' architecture — only the design that best fits the system's specific requirements for scale, consistency, latency, and cost. By understanding common failure patterns and the proven solutions that address them, engineers can build systems that are resilient, performant, and easier to maintain as they grow.